

Building ML web apps with Gradio

Gradio를 활용한 ML 웹 앱 제작

01 나의 첫 Gradio 웹 앱 만들기

목차 01 나의 첫 Gradio 웹 앱 만들기

1. 웹 앱의 개념
2. 웹 앱 제작을 위해 필요한 기술
3. Gradio 소개
4. Gradio의 동작 방식 이해
5. 샘플 코드를 사용하여 첫 번째 Gradio 웹 앱 만들기
6. Gradio 코드 구조의 이해
7. Gradio 활용 목표 및 계획

1. 웹 앱의 개념



Web Application



웹 앱(Web Application)은 사용자의 웹 브라우저에서 실행되는 응용 소프트웨어로,
원격으로 정보를 교환하고, 사용자와 연결될 수 있도록 서비스를 제공



웹 앱과 일반 웹 사이트의 차이

Gradio를 활용한
ML 웹 앱 제작

01 나의 첫 Gradio
웹 앱 만들기

1. 웹 앱의 개념

일반 웹 사이트

Website



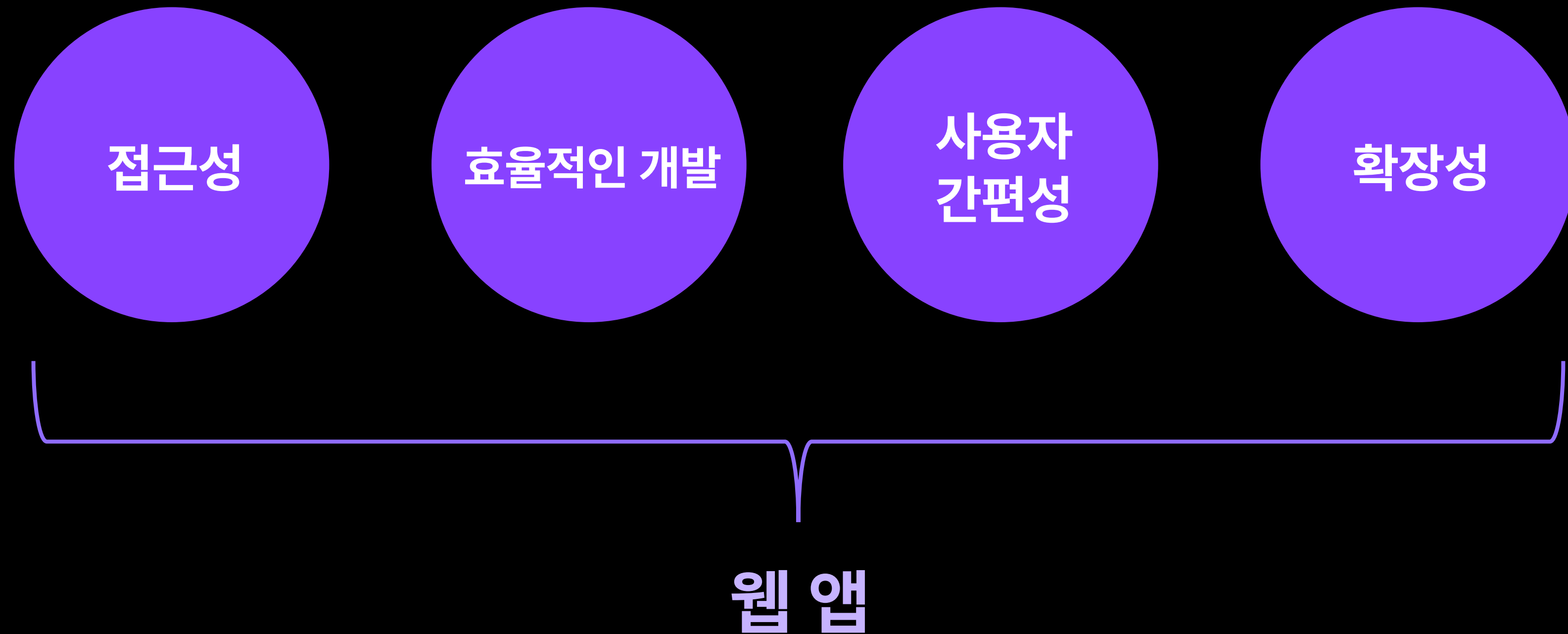
- 사용자에 일방적인 정보제공 중심
- 정적인 콘텐츠에 적합

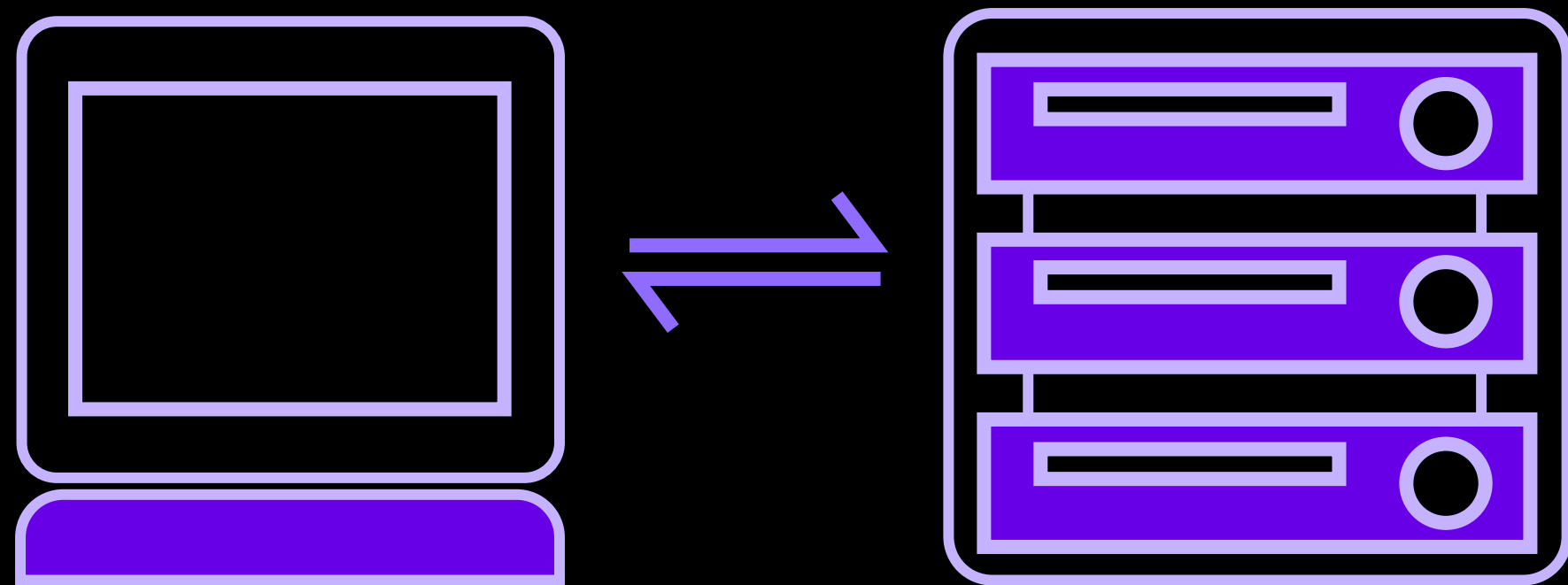
웹 앱

Web App



- 사용자와의 다양한 상호작용 제공
- 동적인 콘텐츠에 적합





클라이언트
Client

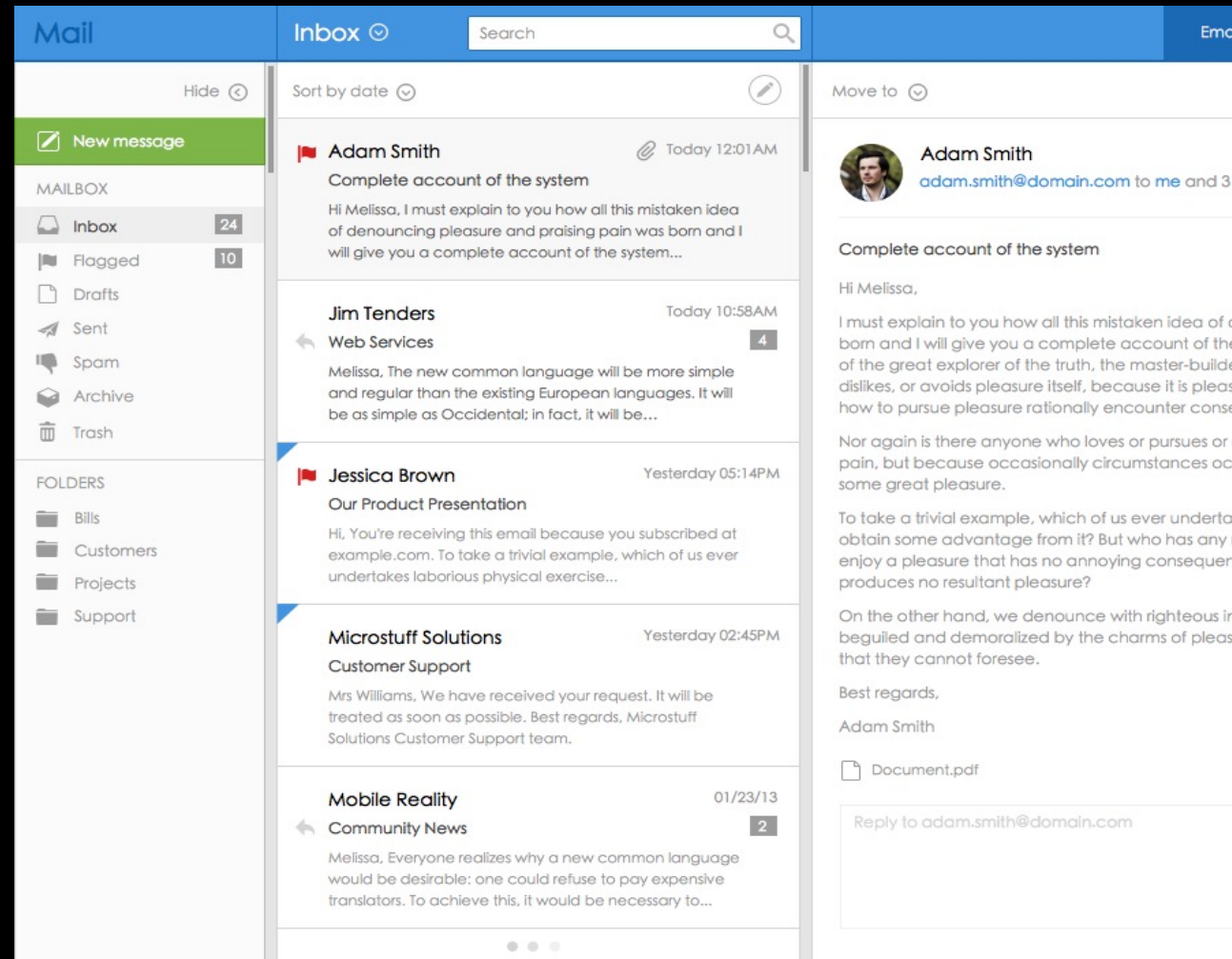
서버
Web App Server

클라이언트 (Client)

- 사용자 인터페이스(UI)를 통해 상호작용하는 부분
- 서버로부터 데이터를 요청 및 수신

서버 (Server)

- 비즈니스 로직 및 데이터 처리
- 클라이언트의 요청에 대한 응답을 생성
- 인증, 보안 등의 작업을 수행



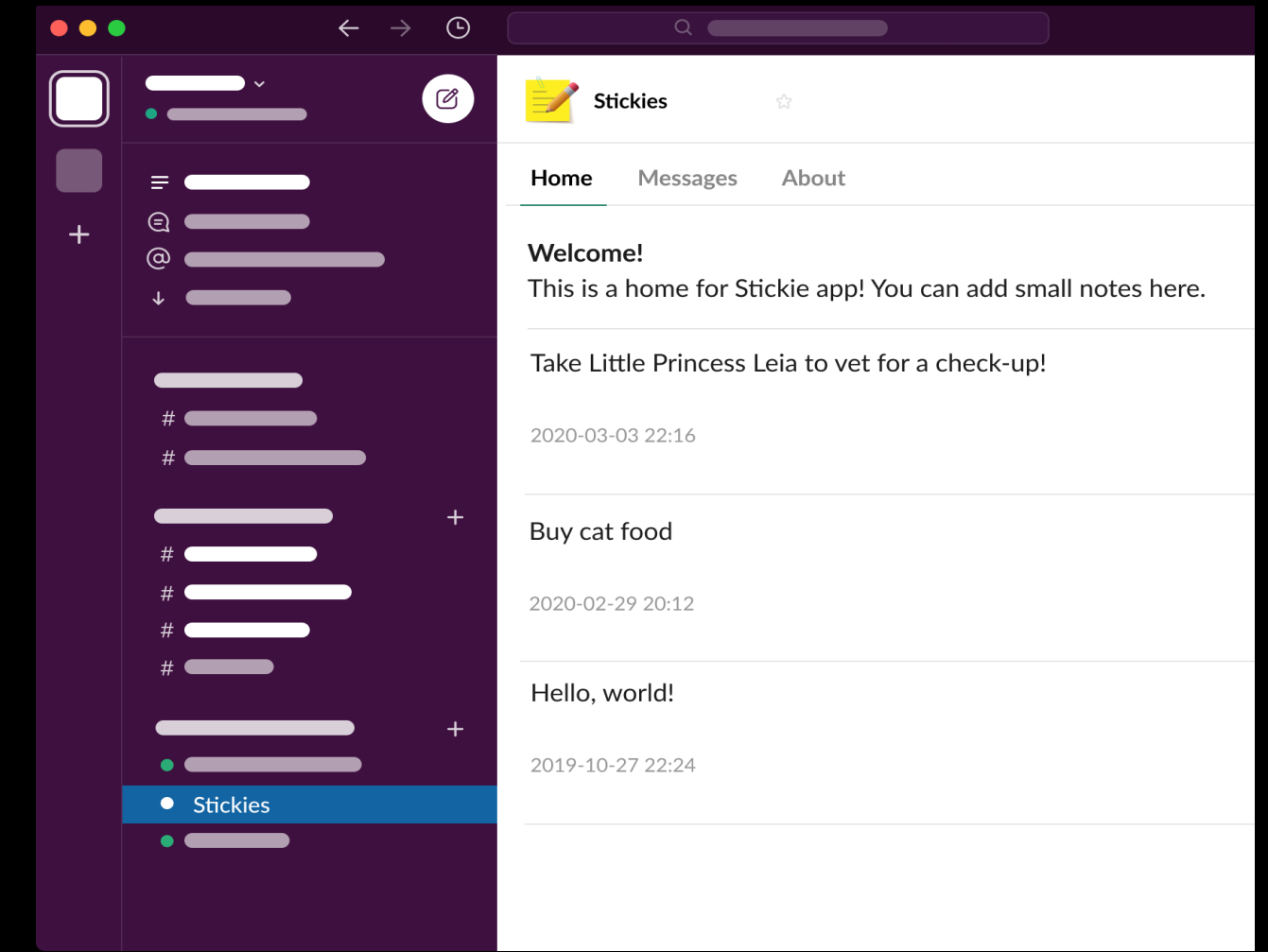
웹 메일

- 구글메일, 네이버메일 등



온라인 전자상거래

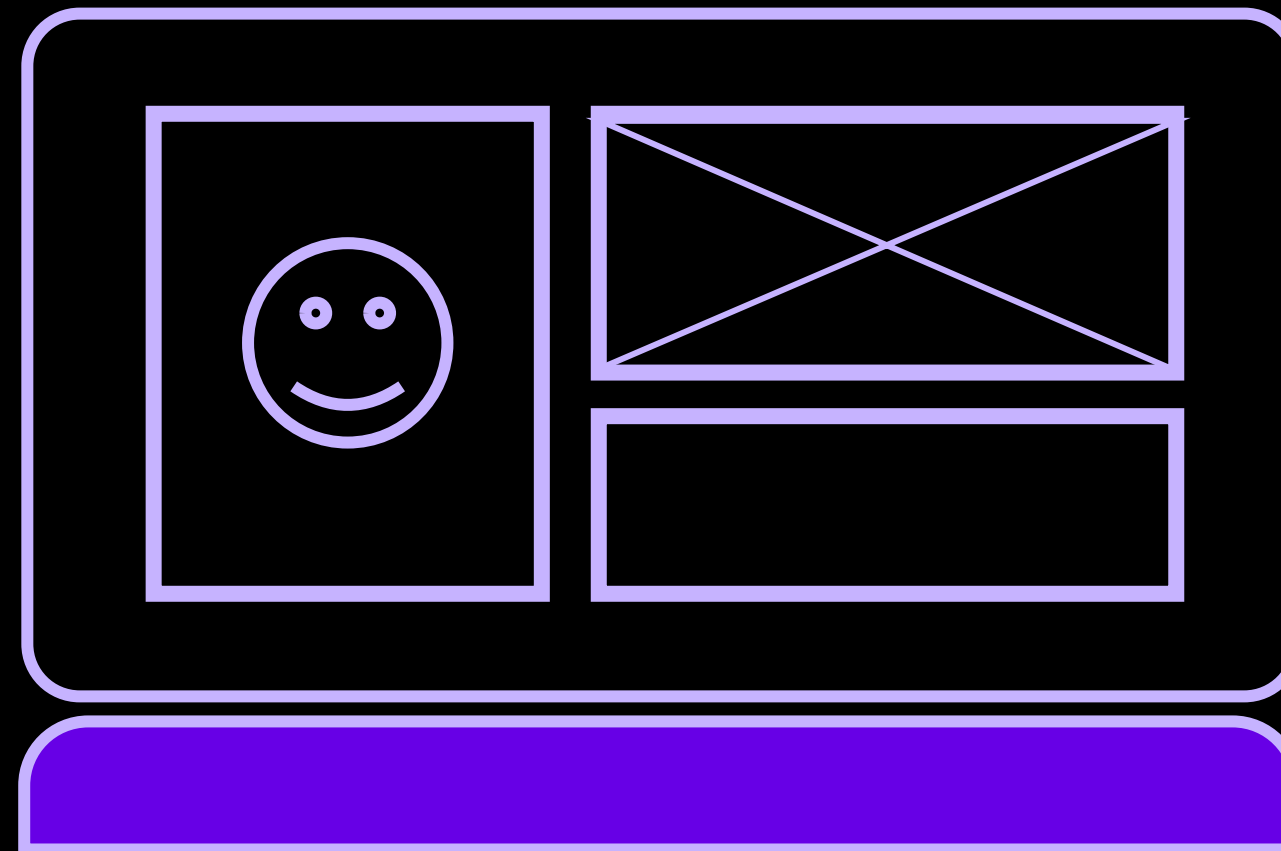
- 무신사, 에이블리 등



협업 툴

- 슬랙, 노션 등

2. 웹 앱 제작을 위해 필요한 기술



Frontend

웹 앱의 **프론트엔드**는 사용자가 브라우저를 통해 직접 상호작용하는 영역을 담당,
사용자 인터페이스, 레이아웃, 디자인 등을 포함



Backend

웹 앱의 백엔드는 데이터 처리 및 저장, 사용자 인증 및 승인 관리,
외부 서비스 및 API와의 통신 등을 담당

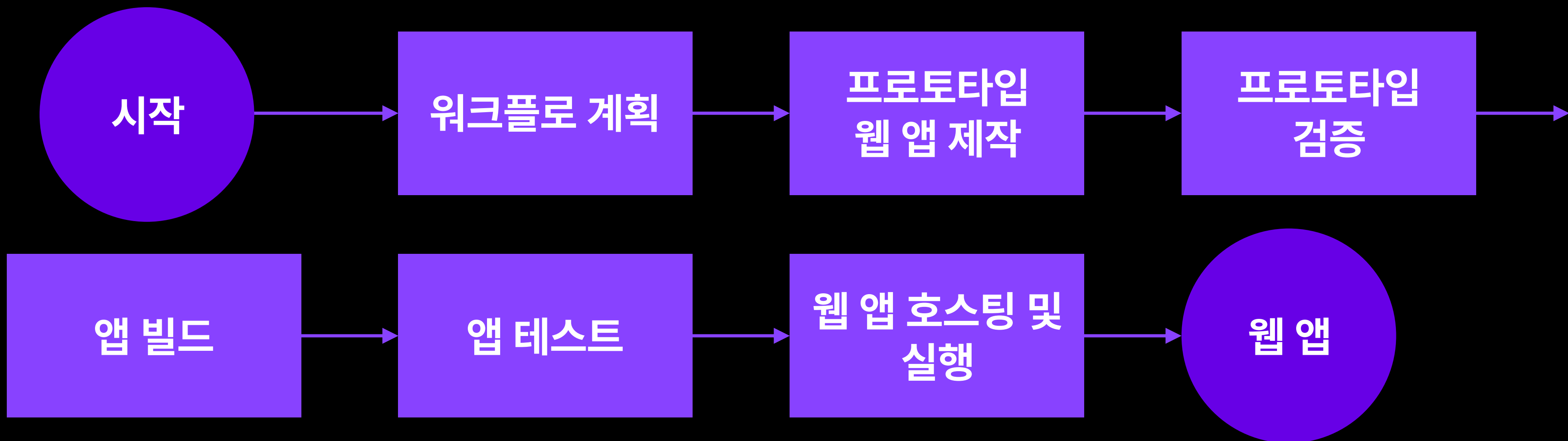


단계별 웹 앱 개발과정

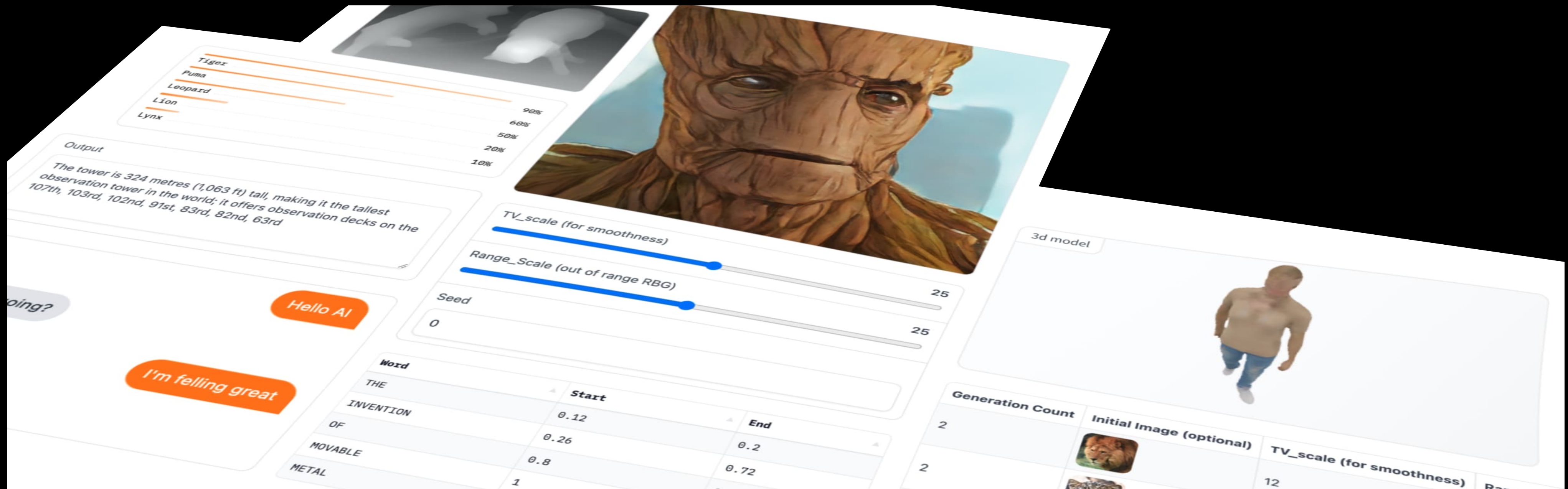
Gradio를 활용한
ML 웹 앱 제작

01 나의 첫 Gradio
웹 앱 만들기

2. 웹 앱 제작을 위해
필요한 기술



3. Gradio 소개



Gradio는 머신러닝 앱 구축 및 공유 서비스로,
편리하고 친숙한 웹 인터페이스를 제공하여 머신러닝 모델을 빠르게 시연할 수 있도록 지원



모델 배포 · 공유

- Gradio는 손쉽게 모델을 배포하고, 그 결과를 공유할 수 있도록 지원
- 배포된 모델을 다양한 사용자들이 활용 및 테스트 가능

다양한 프레임워크 지원

- 다양한 딥러닝 및 머신러닝 프레임워크와 통합, 목적에 맞게 활용 가능
- TensorFlow, PyTorch, Scikit-learn 등을 사용하는 모델을 지원
- HuggingFace 등과 연동하여 다양한 사전훈련 모델들을 공유 가능

직관적인 인터페이스 제공

- 간단한 코드의 사용자 친화적인 인터페이스로 모델을 감싸는 기능 제공
- 텍스트, 이미지, 오디오, 비디오 등 다양한 유형의 데이터 적용 가능
- 이를 통해 개발자들은 모델을 쉽게 시각화하고 상호작용할 수 있음

상호작용 지원

- 사용자의 필요에 따라 모델의 입력값을 조정하고 결과 시각화가 가능
- 모델 테스트 결과 및 사용자 피드백을 수용하여 모델개선이 용이하도록 지원

Streamlit

Streamlit

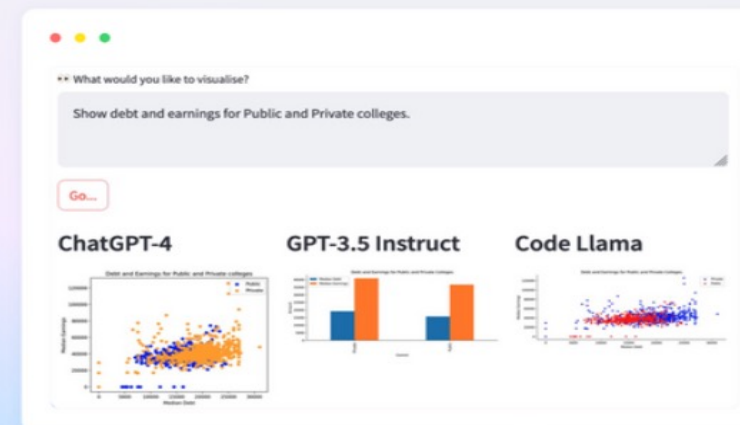
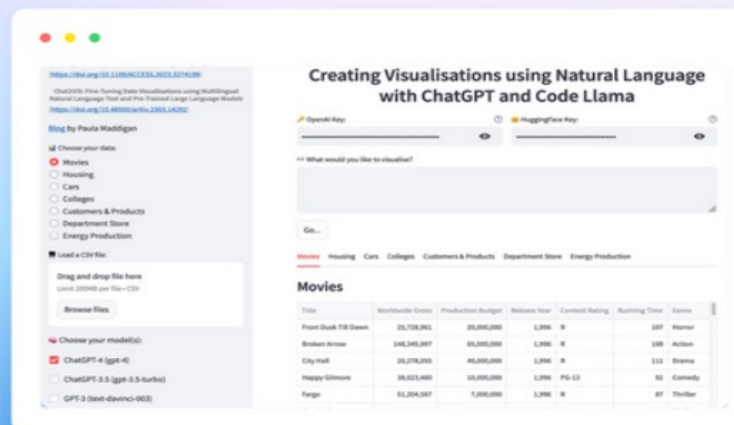
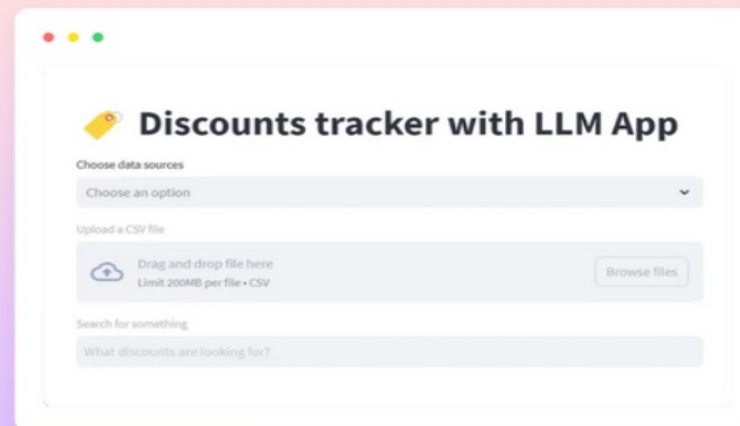
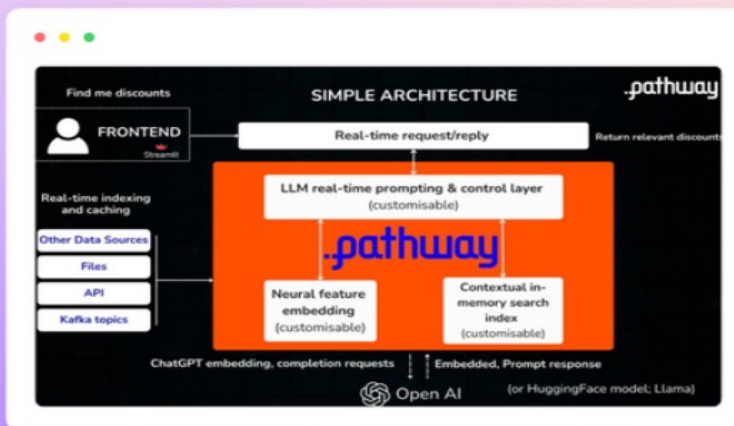
- 데이터과학 및 기계학습을 위한 맞춤형 웹 앱을 구축하는 오픈소스 라이브러리

[장점]

- 앱의 모양과 테마를 손쉽게 변경 가능
- 각 기능에 대한 예시 자료가 풍부

[단점]

- Streamlit 자체구문에 대한 약간의 숙련도 필요
- 제한된 위젯 세트 제공, Jupyter Notebook과 통합되지 않음
- 데이터 업로드 유형 및 용량 제한(50MB) 존재



Dash

Dash

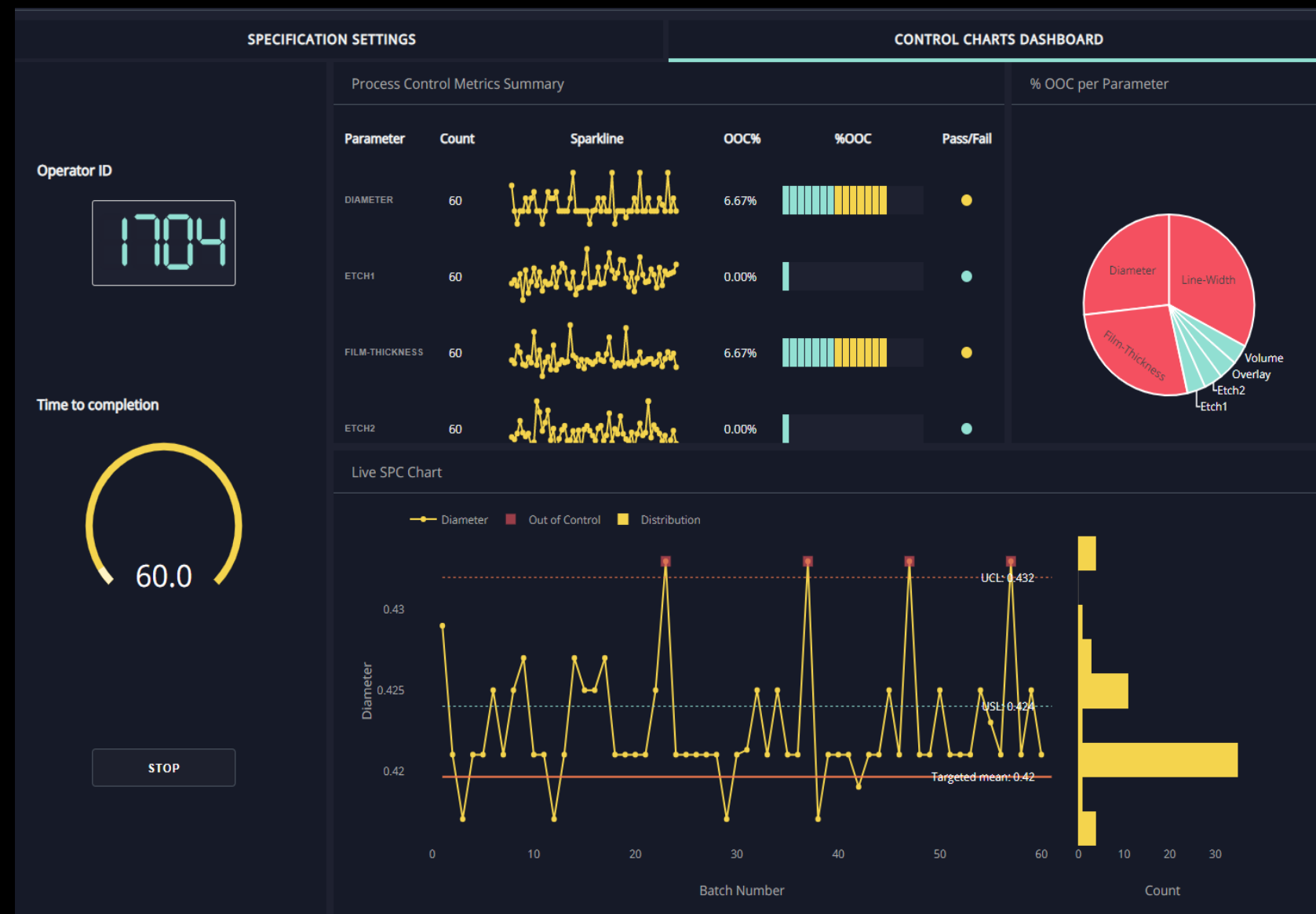
- 반응형 웹 앱을 만들 수 있는 오픈소스 라이브러리

[장점]

- 데이터 탐색, 데이터 분석, 모델링, 시각화, 기기 제어, 보고와 같은 영역에 특화
- 커스터마이징이 용이

[단점]

- 엔터프라이즈 시장에 더 중점을 두어 오픈소스 버전에서는 모든 기능을 포함하지 않음
- Gradio, Streamlit에 비해 더 많은 코드작성 필요
- 기능적 구현의 난이도가 상대적으로 높은 편



4. Gradio의 동작 방식 이해

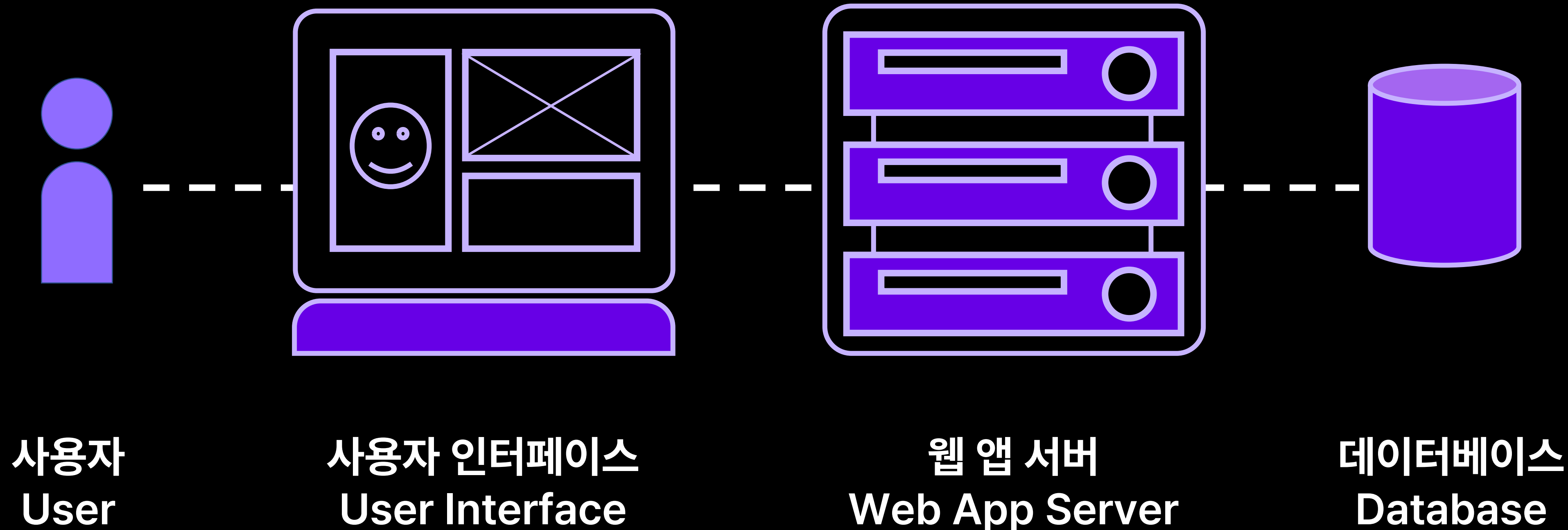


웹 앱의 작동구조

Gradio를 활용한
ML 웹 앱 제작

01 나의 첫 Gradio
웹 앱 만들기

4. Gradio의
동작 방식 이해





웹 앱 서버의 역할

Gradio를 활용한
ML 웹 앱 제작

01 나의 첫 Gradio
웹 앱 만들기

4. Gradio의
동작 방식 이해



모델 배포

- 사전훈련된 모델의 서비스
- 프레임워크 및 시스템 통합



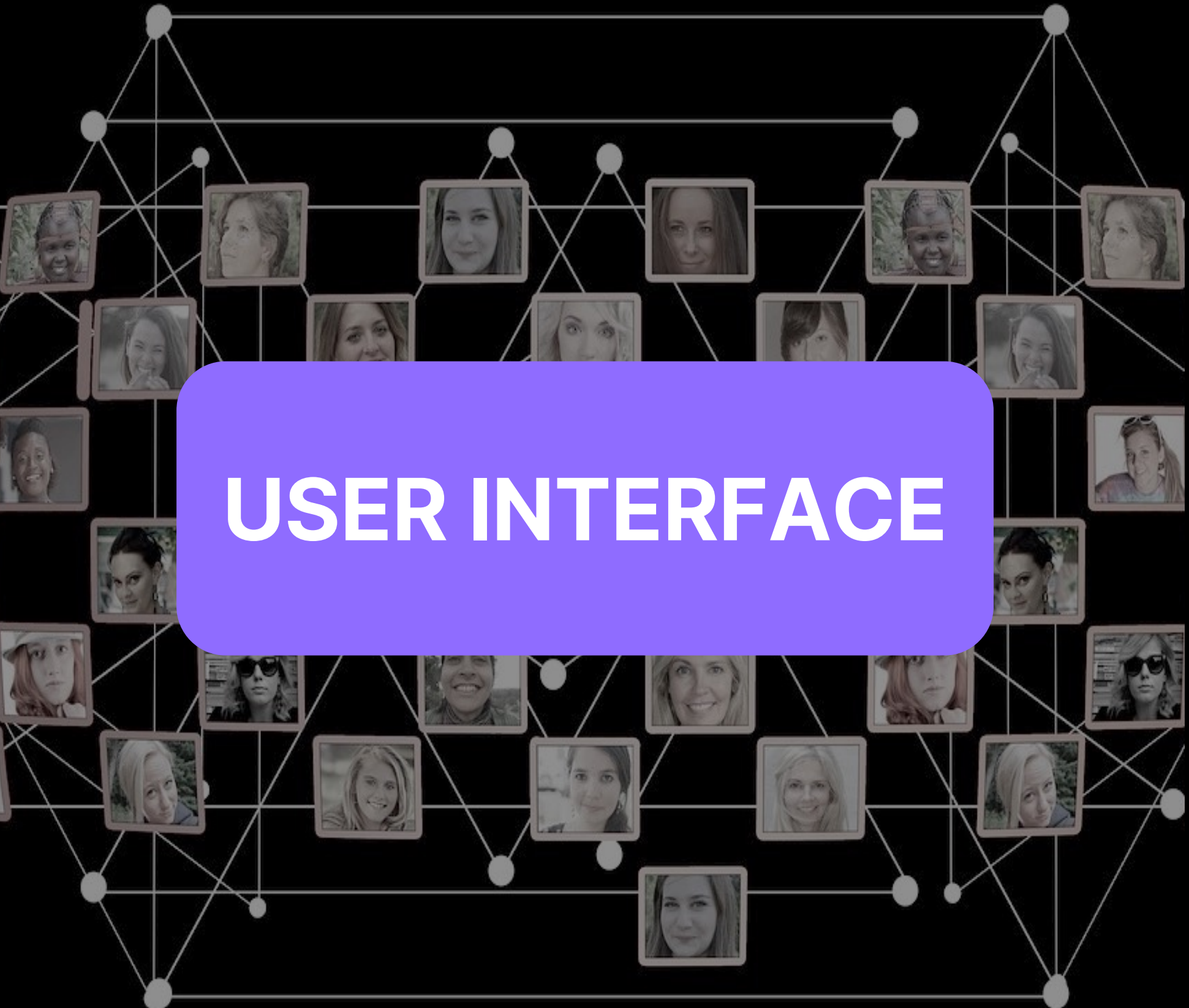
웹 앱 로직처리

- 사용자 입력의 전달 및 처리
- 예측 및 분석결과 도출



인터페이스 관리

- 다양한 입력유형 지원
- 대화형 대시보드 생성 등



USER INTERFACE

- 사용자 맞춤형 입력 폼 제공
- 인터랙티브 기능 지원
- 다양한 출력형태 및 시각화 도출
- 사용자 경험 향상

5. 샘플 코드를 사용하여 첫 번째 Gradio 웹 앱 만들기



Quickstart



Docs



Guides



Playground^{NEW}



Community



Gradio에서 제공하는 샘플코드(Quick Start)를 사용하여, 손쉽게 웹 앱 구현 가능

Python 3.8 이상이 설치되어 있으면 끝, 다른 것은 필요 없음



1. Hello, World

- 사용자가 이름(000)을 입력하면, "Hello, 000"라고 응답해주는 간단한 기능 구현
- 쥬피터노트북(Jupyter Notebook)에서도 실행 가능함

1) Gradio 설치하기

- pip를 사용하여 gradio 설치

```
pip install gradio
```




1. Hello, World

2) 샘플코드 실행

- 아래 코드를 Python 스크립트 또는 Jupyter Notebook 등에서 실행

```
import gradio as gr

def greet(name):
    return "Hello " + name + "!"

demo = gr.Interface(fn=greet, inputs="text", outputs="text")

demo.launch()
```



1. Hello, World

3) 사용자 입력 및 결과확인

예시 이미지

The image shows a Gradio web application interface. It consists of two main columns. The left column has a text input field labeled 'name' and a 'Submit' button. Below the input field are two buttons: 'Clear' and 'Submit'. The right column has a text output field labeled 'output' and a 'Flag' button. The 'Submit' button is orange, while the others are grey.



2. 다중 입력 및 출력

- 이름 이외에도 추가적인 입력과 출력이 가능하도록 기능 구현
- 앞선 과정에서의 gradio 설치를 전제로 함

1) 입출력 요소

- 이름(name); 사용자 입력
- 아침 여부(is_morning) 선택; 체크박스 형식
- 기온(temperature) 지정; 슬라이더 형식

The image shows a Gradio web application interface. On the left, there are three input components: a text box labeled 'name', a checkbox labeled 'is_morning', and a slider labeled 'temperature' with a value of 0. Below these are two buttons: 'Clear' and 'Submit'. On the right, there are two output components: a text box labeled 'output 0' and a text box labeled 'output 1' with the value 0. At the bottom right is a button labeled 'Flag'.



2. 다중 입력 및 출력

2) 함수정의 및 구성요소의 전달

```
import gradio as gr

def greet(name, is_morning, temperature):
    salutation = "Good morning" if is_morning else "Good evening"
    greeting = f"{salutation} {name}. It is {temperature} degrees today"
    celsius = (temperature - 32) * 5 / 9
    return greeting, round(celsius, 2)

demo = gr.Interface(
    fn=greet,
    inputs=["text", "checkbox", gr.Slider(0, 100)],
    outputs=["text", "number"],
)
demo.launch()
```



2. 다중 입력 및 출력

3) 사용자 입력 및 선택, 결과확인

예시 이미지

The image shows a Gradio web application interface with a dark theme. On the left, there are three input components: a text box labeled 'name', a checkbox labeled 'is_morning', and a slider labeled 'temperature' with a numeric display showing '0'. Below these inputs are two buttons: 'Clear' and 'Submit'. On the right, there are two output text boxes labeled 'output 0' and 'output 1'. At the bottom right, there is a button labeled 'Flag'.



3. 이미지 변환

- Gradio는 이미지, 데이터프레임, 동영상, 라벨 등 다양한 형태의 구성요소를 지원
- 이미지를 입력으로 넣으면 다른 분위기로 변환해주는 기능 구현

1) 이미지 변환 유형

- 세피아(sepia) 톤으로 변환
 - 세피아 필터의 설정; 빨강(R), 녹색(G), 파랑(B)의 각 채널에 대해 다른 가중치를 부여
 - 세피아 필터 적용 및 변환
 - 픽셀값의 정규화



3. 이미지 변환

2) 함수정의 및 구성요소의 전달

```
import numpy as np
import gradio as gr

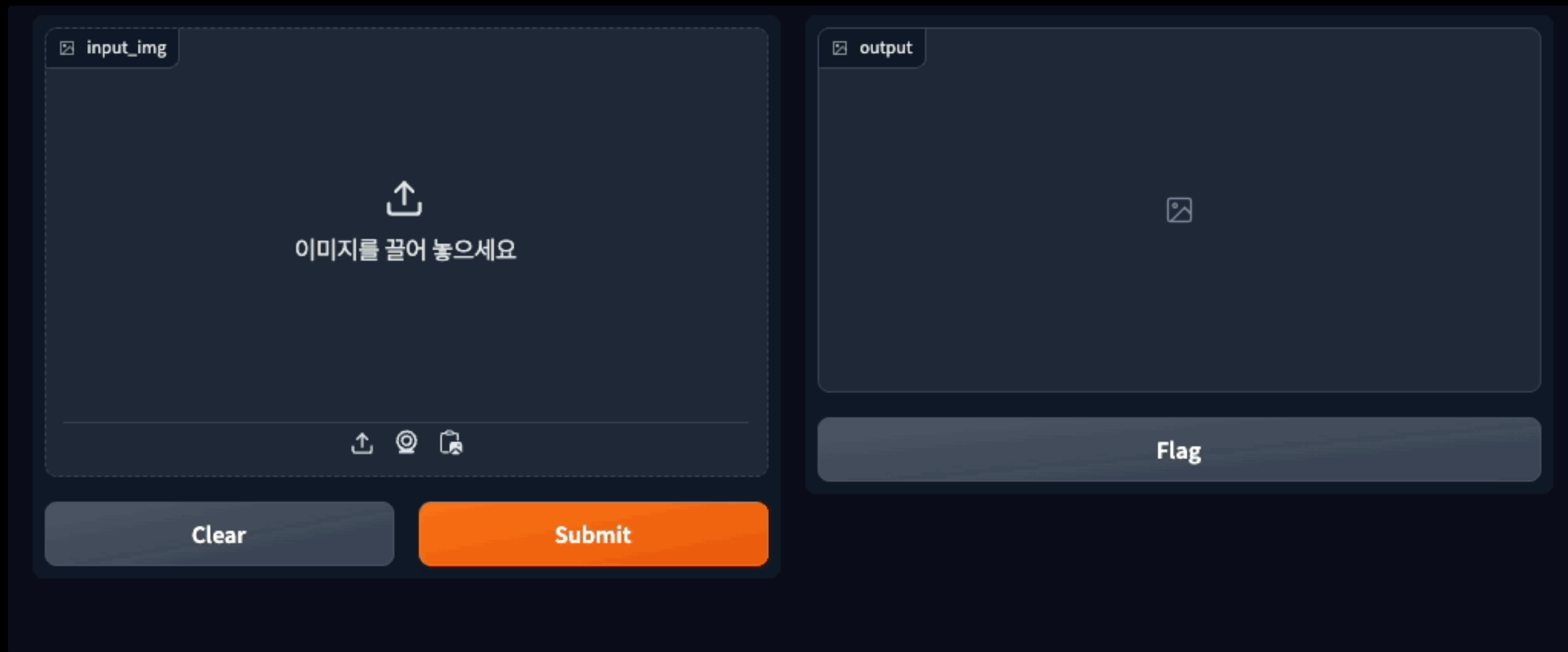
def sepia(input_img):
    sepia_filter = np.array([
        [0.393, 0.769, 0.189],
        [0.349, 0.686, 0.168],
        [0.272, 0.534, 0.131]
    ])
    sepia_img = input_img.dot(sepia_filter.T)
    sepia_img /= sepia_img.max()
    return sepia_img

demo = gr.Interface(sepia, gr.Image(), "image")
demo.launch()
```

3. 이미지 변환

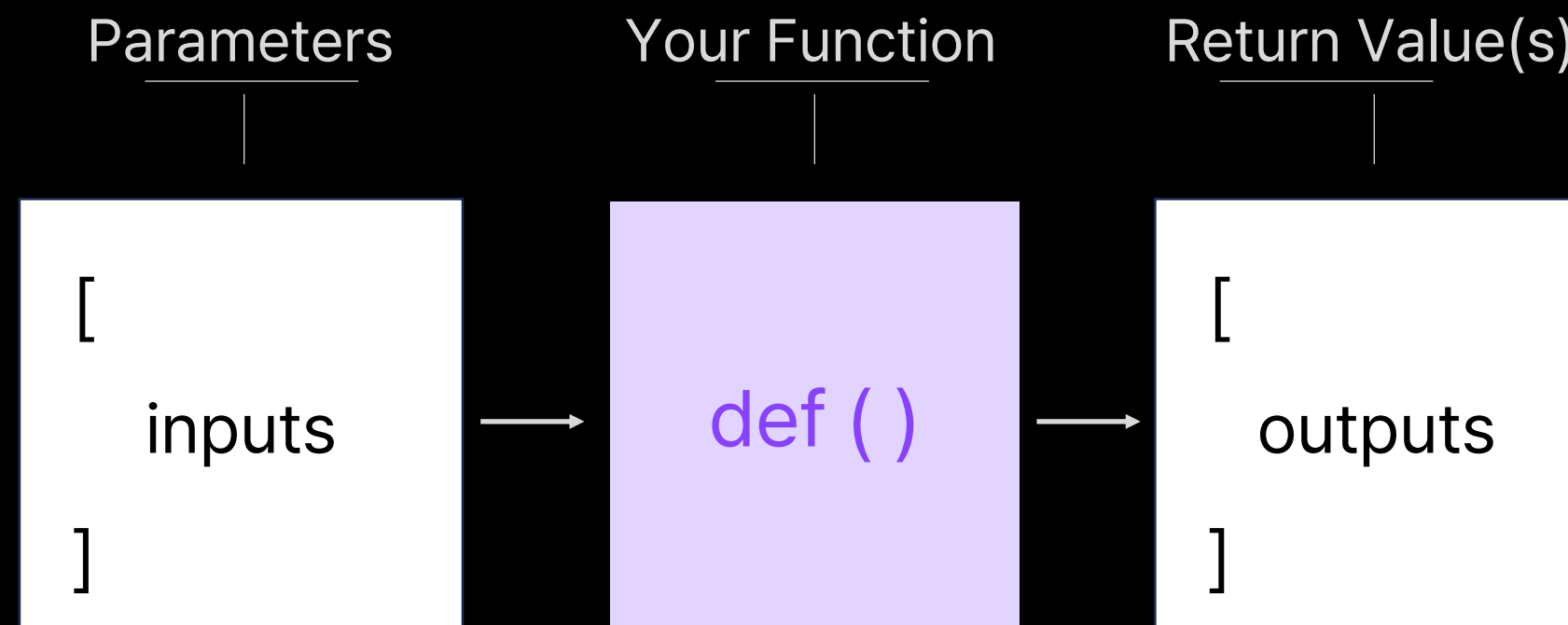
3) 사용자 입력 및 결과확인

예시 이미지



The image shows a Gradio web application interface for image transformation. It features two main components: an input area on the left and an output area on the right. The input area, labeled 'input_img', contains a large dashed box with a central upload icon and the text '이미지를 끌어 놓으세요' (Drag and drop image here). Below this box are three small icons: a file upload icon, a camera icon, and a gallery icon. At the bottom of the input area are two buttons: 'Clear' and 'Submit'. The output area, labeled 'output', is a large empty box with a small image icon in the center. Below the output box is a 'Flag' button. The entire interface is set against a dark background with light gray text and icons.

6. Gradio 코드 구조의 이해



0. 함수를 사용한 입출력 처리

- 제작된 함수에 대한 입출력을 손쉽게 웹 앱으로 전달하는 것이 주 목적
- 대표적으로 인터페이스(Interface), 블록(Block) 등의 구성 요소를 사용하여, 함수의 입출력을 처리

1. 인터페이스 클래스

- 앞선 샘플코드는 **인터페이스 클래스(`gr.Interface`)**를 만들어 사용
- 인터페이스 클래스에는 구현하고자 하는 어떠한 함수이든 입력 가능

1) 인터페이스 클래스(`Interface class`)의 핵심 매개변수

- **fn** : 구현하고자 하는 함수(function)
- **inputs** : 입력으로 넣을 구성요소들 ("text", "image", "audio" 등)
- **outputs** : 출력으로 사용할 구성요소들 ("text", "image", "label" 등)

1. 인터페이스 클래스

2) 주요 구성요소(Components)

- 텍스트 박스(Textbox) : 텍스트를 입력
- 체크 박스(Checkbox) : 해당여부를 체크
- 드롭다운(Dropdown) : 보기/옵션 선택
- 슬라이더(Slider) : 최소~최대 값 사이 선택



The image displays four Gradio interface components in a row:

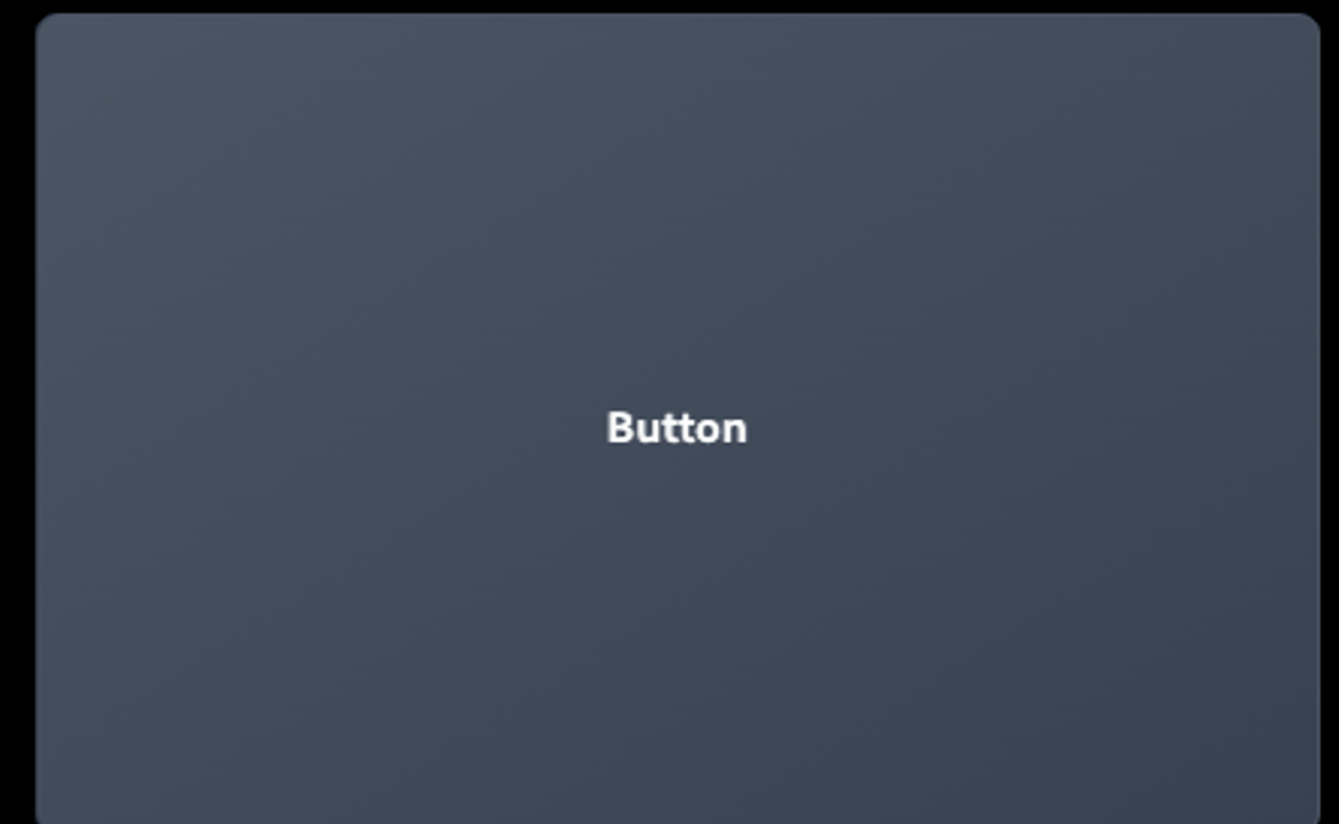
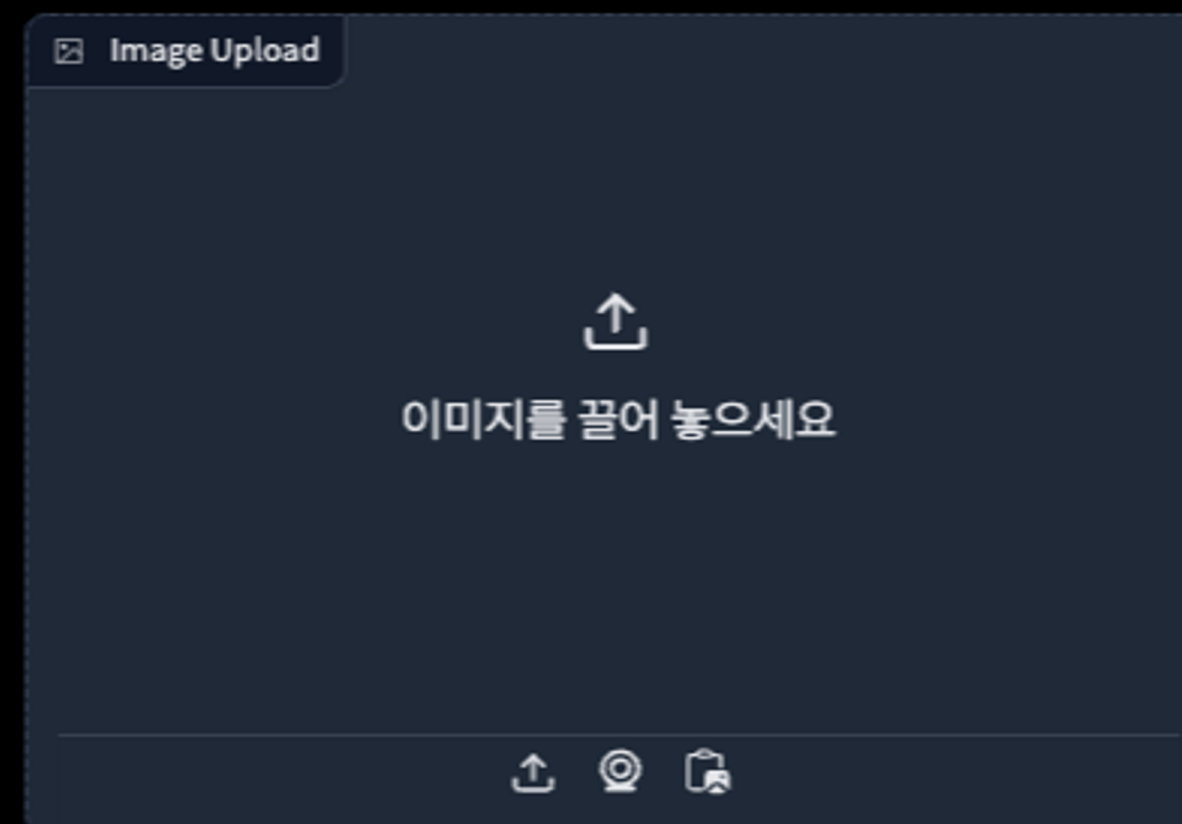
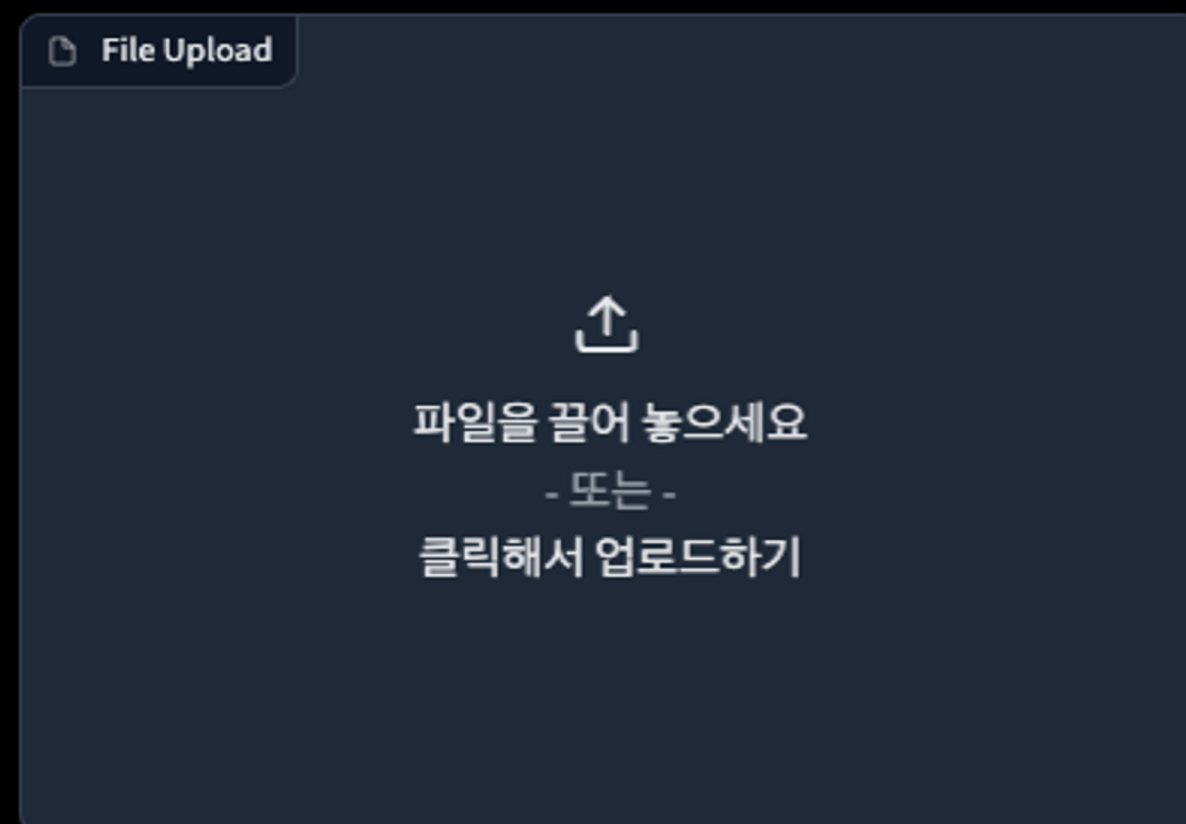
- Tesxtbox**: A text input field with the placeholder text "여기에 텍스트를 입력".
- Checkbox**: A checked checkbox with the label "Checkbox".
- Dropdown**: A dropdown menu with a list of options: "a", "b", and "c".
- Slider**: A slider control with a value of 24.



1. 인터페이스 클래스

2) 주요 구성요소(Components)

- 파일 업로드(File) : 파일을 입력
- 이미지 업로드(Image) : 이미지를 입력
- 버튼(Button) : 클릭하여 동작



1. 인터페이스 클래스

2) 인터페이스 클래스(Interface class) 사용 예시

```
import gradio as gr ..... "gr"로 축약하여 사용

def greet(name):
    return "Hello " + name + "!"
} 함수 정의

demo = gr.Interface(
    fn=greet,
    inputs=gr.Textbox(lines=2, placeholder="Name Here..."),
    outputs="text",
) ..... 인터페이스 클래스
                                     (매개변수
                                     : fn, inputs, outputs)
                                     구성요소 및 속성 정의
demo.launch() ..... demo를 웹에 서빙
```

2. 인터페이스 상태관리(Interface State)

- Gradio에서 상태관리는 **글로벌 방식, 세션 방식**으로 구분됨

1) 글로벌 방식(Global State)

- 모든 함수 호출 및 모든 사용자가 액세스할 수 있어야 할 때
- 함수 외부에서 대규모 모델을 로드하고, 함수 내부에서 사용하고자 할 때

2) 세션 방식(Session State)

- 페이지 세션 내에서는 지속되나, 다른 사용자 간에는 공유되지 않아야 할 때
- 전후 사용자의 모델사용 기록과 관계없이 독립적인 처리가 필요할 때



2. 인터페이스 상태관리(Interface State)

1) 글로벌 방식(Global State)

- 다음 코드에서 **scores**는 모든 사용자 간에 공유, 여러 사용자의 기록 중 상위 3개 점수가 수집

```
import gradio as gr

scores = []

def track_score(score):
    scores.append(score)
    top_scores = sorted(scores, reverse=True)[:3]
    return top_scores

demo = gr.Interface(
    track_score,
    gr.Number(label="Score"),
    gr.JSON(label="Top Scores")
)
demo.launch()
```



2. 인터페이스 상태관리(Interface State)

2) 세션 방식(Session State)

- 챗봇은 세션 방식이 필요한 웹 앱의 유형 중 하나
- 다른 사용자 간의 채팅 기록이 섞이지 않도록 **동일 세션** 내에서만 데이터가 지속
- 사용자가 바뀌어 세션이 달라지면 초기화

''' 세션 상태에 데이터 저장하기 '''

인터페이스 상태를 표시하는 추가 매개변수를 함수에 전달

함수의 실행완료 시에 업데이트된 상태 값을 추가 반환 값으로 반환(return)

인터페이스를 생성할 때, 상태 입력 및 출력 구성요소를 추가

1. 블록 구조

- 코드 작성 시 **블록(gr.Blocks)**을 만들어 더 자유롭게 사용 가능
- 블록 안에 생성된 모든 구성요소는 자동으로 앱에 추가됨(입력 순서대로)
- 구성요소의 제어, 데이터 처리 가능, 사용자와 상호작용 이벤트 등을 수행

[참고] 이벤트(Event)

- 사용자 인터페이스에서 발생하는 다양한 이벤트(Event)를 감지, 특정 동작을 수행하도록 지원
- 버튼 클릭, 텍스트 입력, 파일 업로드 등
- 이러한 기능을 활용하여 사용자의 동작에 따른 더욱 동적이고 유연한 인터페이스를 구현할 수 있음

1. 블록 구조

1) 블록 코드 예시

- With 구문으로 작성 , 텍스트박스 및 버튼 추가

```
import gradio as gr

def greet(name):
    return "Hello " + name + "!"

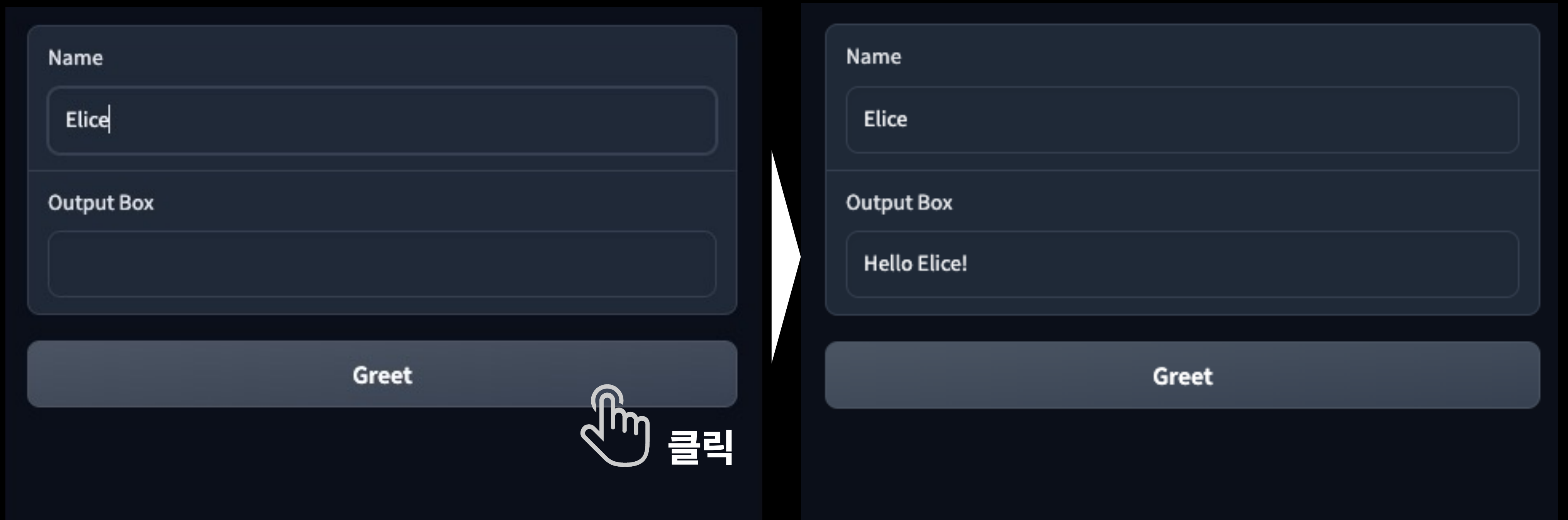
with gr.Blocks() as demo:
    name = gr.Textbox(label="Name")
    output = gr.Textbox(label="Output Box")
    greet_btn = gr.Button("Greet")
    greet_btn.click(fn=greet, inputs=name, outputs=output, api_name="greet")

demo.launch()
```

블록 구조
("Greet" 버튼 클릭 시
이벤트 발생)

1. 블록 구조

2) 블록 코드 실행 예시





2. 레이아웃 제어

- 기본적으로 수직 정렬되는 블록의 구성요소를 재배열, 원하는 레이아웃 구성 가능

1) 행(Row)

- with gr.Row : 가로로 나란히 표시

2) 열(Column)

- with gr.Column : 세로로 표시, 수직정렬이 기본이므로 열(Column)은 대부분 행(Row) 안에 중첩
scale 인자로 너비 배율조정 가능 (예, scale=2)
min_width 인자로 최소 너비 설정 (예, min_width=600)



2. 레이아웃 제어

3) 행(Row)과 열(Column) 코드 예시

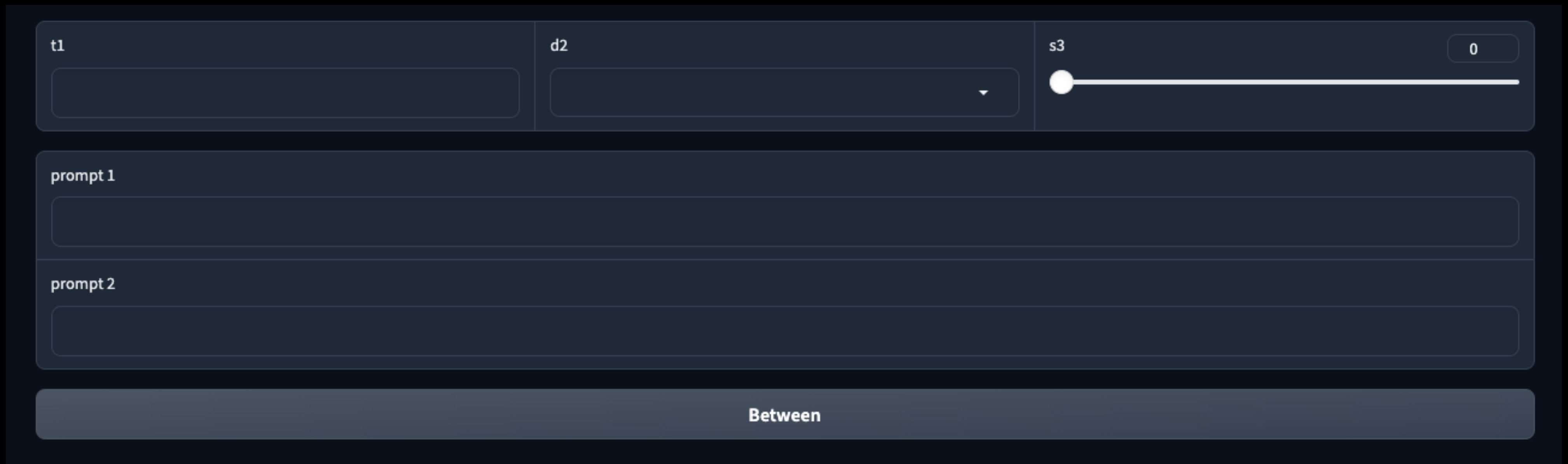
```
import gradio as gr

with gr.Blocks() as demo:
    with gr.Row():
        text1 = gr.Textbox(label="t1")
        drop2 = gr.Dropdown(["a", "b", "c"], label="d2")
        slider3 = gr.Slider(label="s3")
    with gr.Row():
        with gr.Column(scale=1, min_width=600):
            text1 = gr.Textbox(label="prompt 1")
            text2 = gr.Textbox(label="prompt 2")
            inbtw = gr.Button("Between")

demo.launch()
```

2. 레이아웃 제어

3) 행(Row)과 열(Column) 코드 실행 예시



The image displays a Gradio interface with a dark theme. At the top, there are three components arranged horizontally: a text input labeled 't1', a dropdown menu labeled 'd2', and a slider labeled 's3' with a value of 0. Below these, there are two text input fields labeled 'prompt 1' and 'prompt 2'. At the bottom, there is a large button labeled 'Between'.

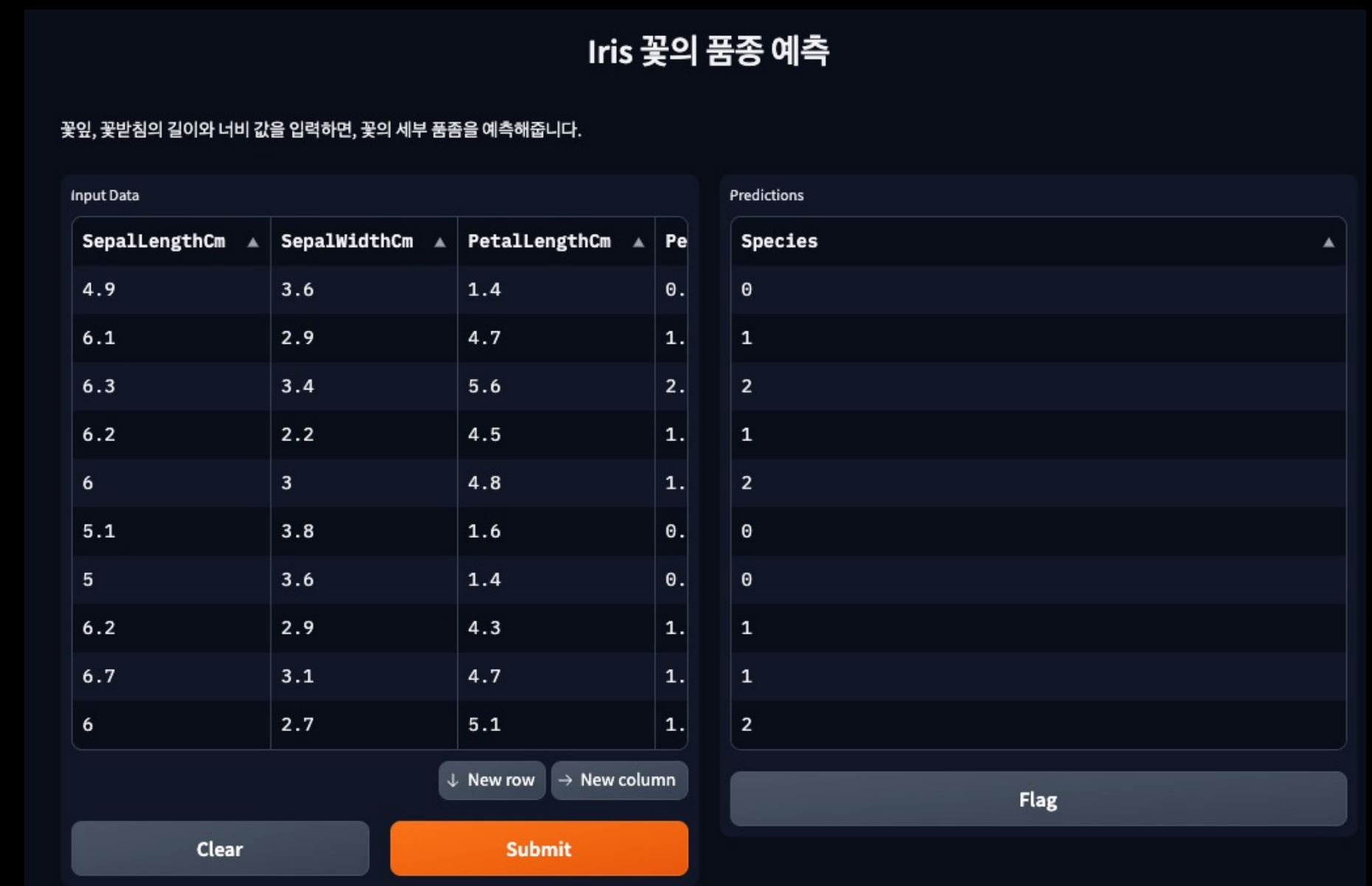
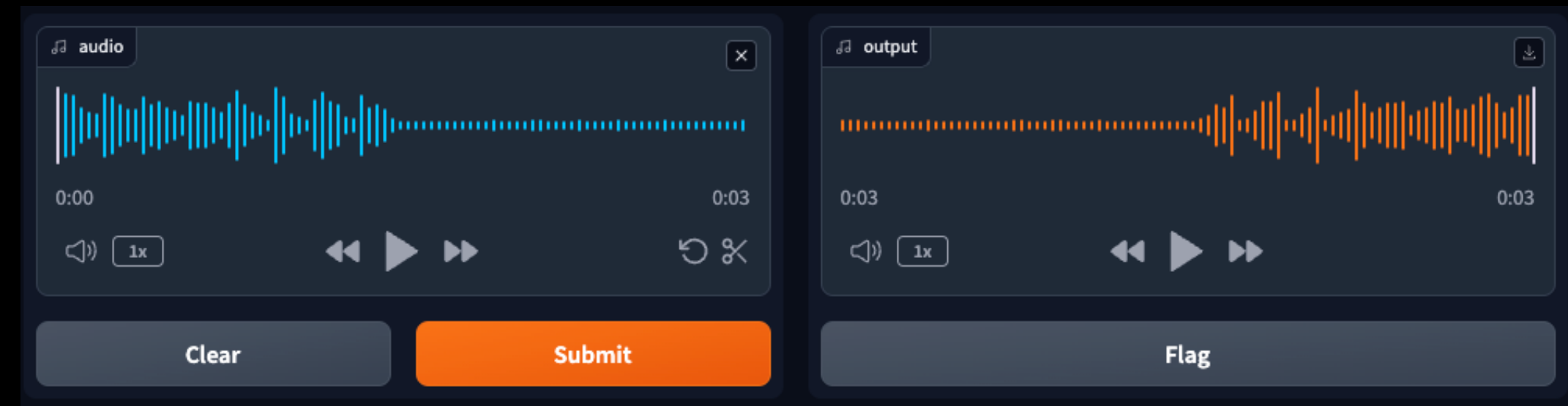
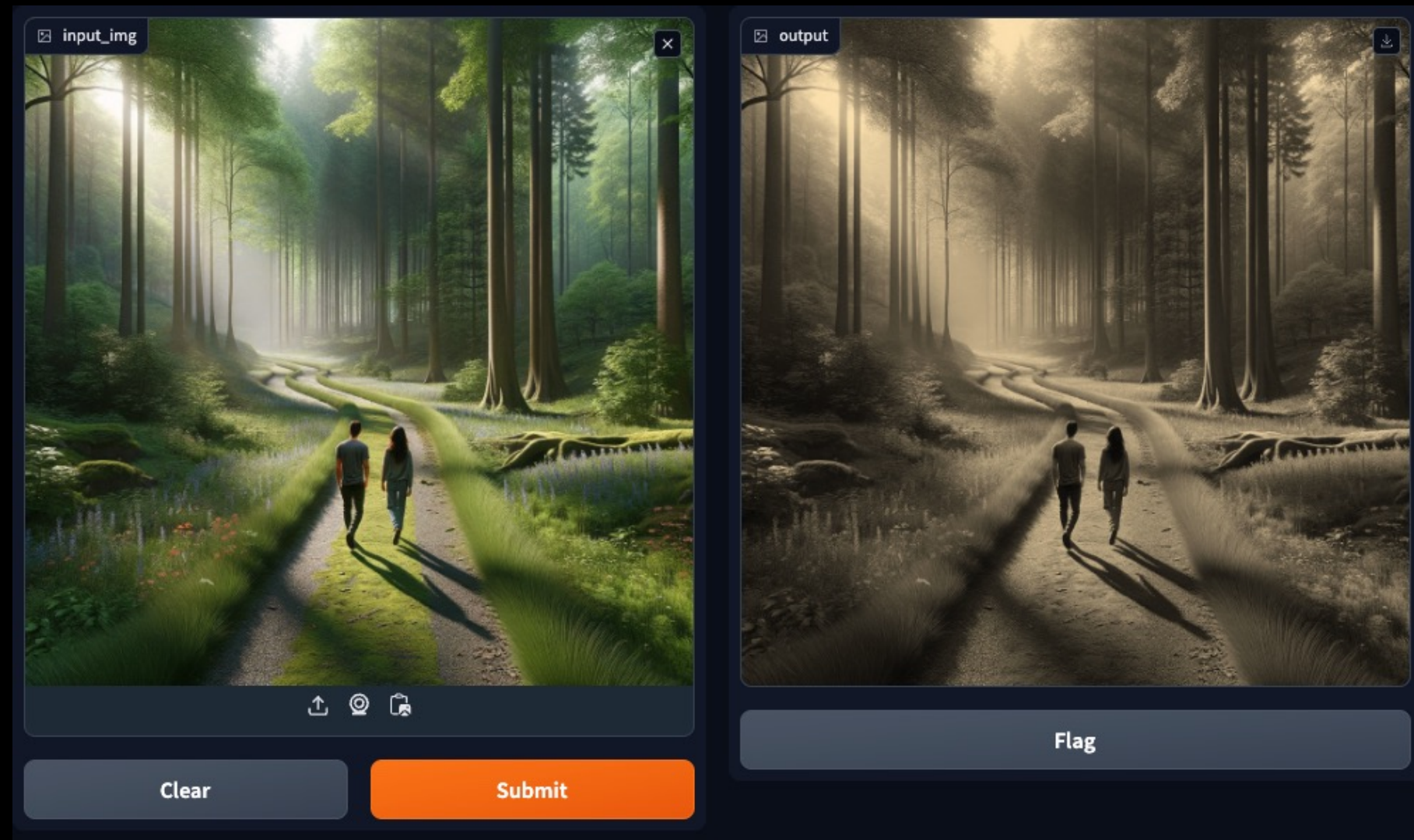


	인터페이스(Interface)	블록(Block)
장점	• 보다 손쉽고 직관적으로 빠른 구현 • 함수를 표현하는데 필요한 대부분의 요소 보유	• 보다 사용자화 할 수 있는 요소가 많음 • 화면 상의 UI를 더 자유롭게 구성 가능
단점	• 화면 구성이 일부 자유롭지 않을 수 있음 • 특정 기능은 사용할 수 없음 (예: session state 변수를 하나만 사용 가능)	• 보다 구체적인 구상이 필요 • 코드가 더 길어짐

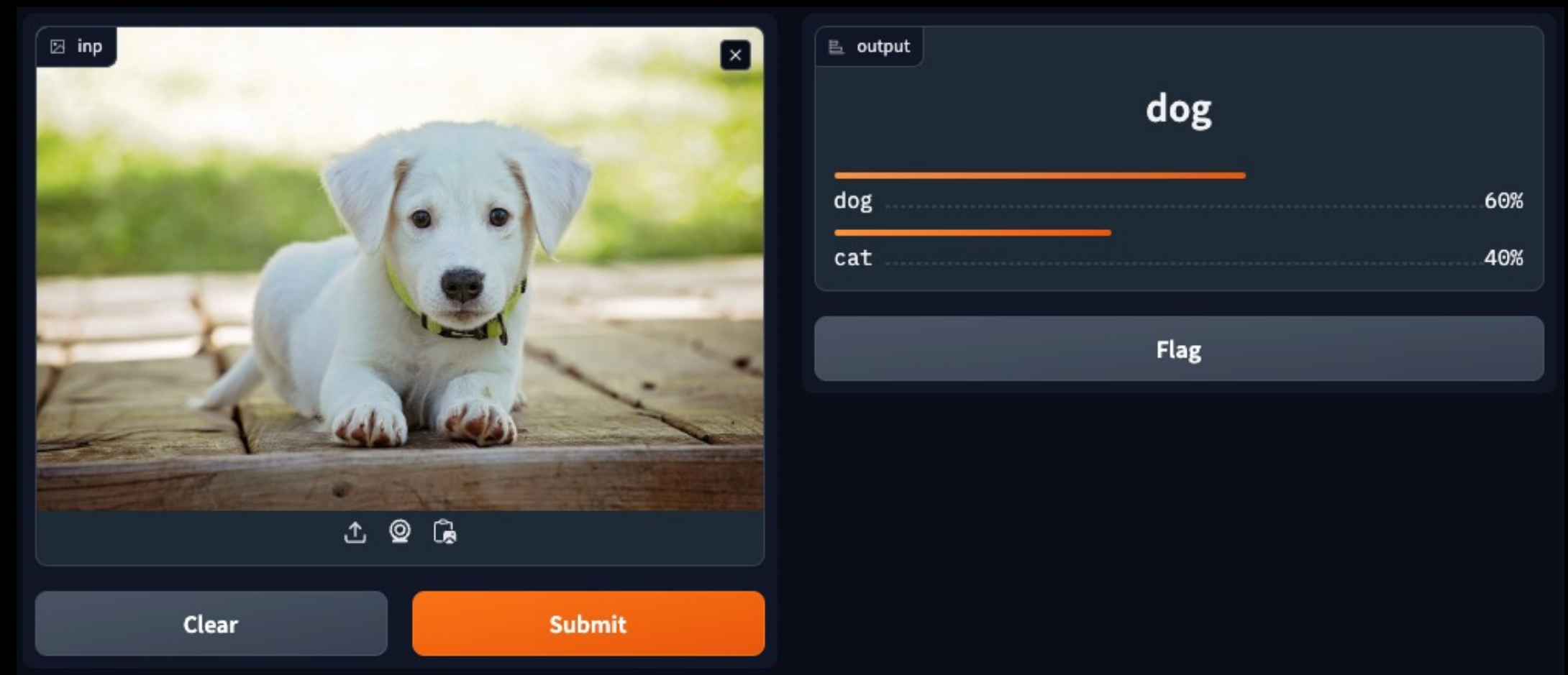
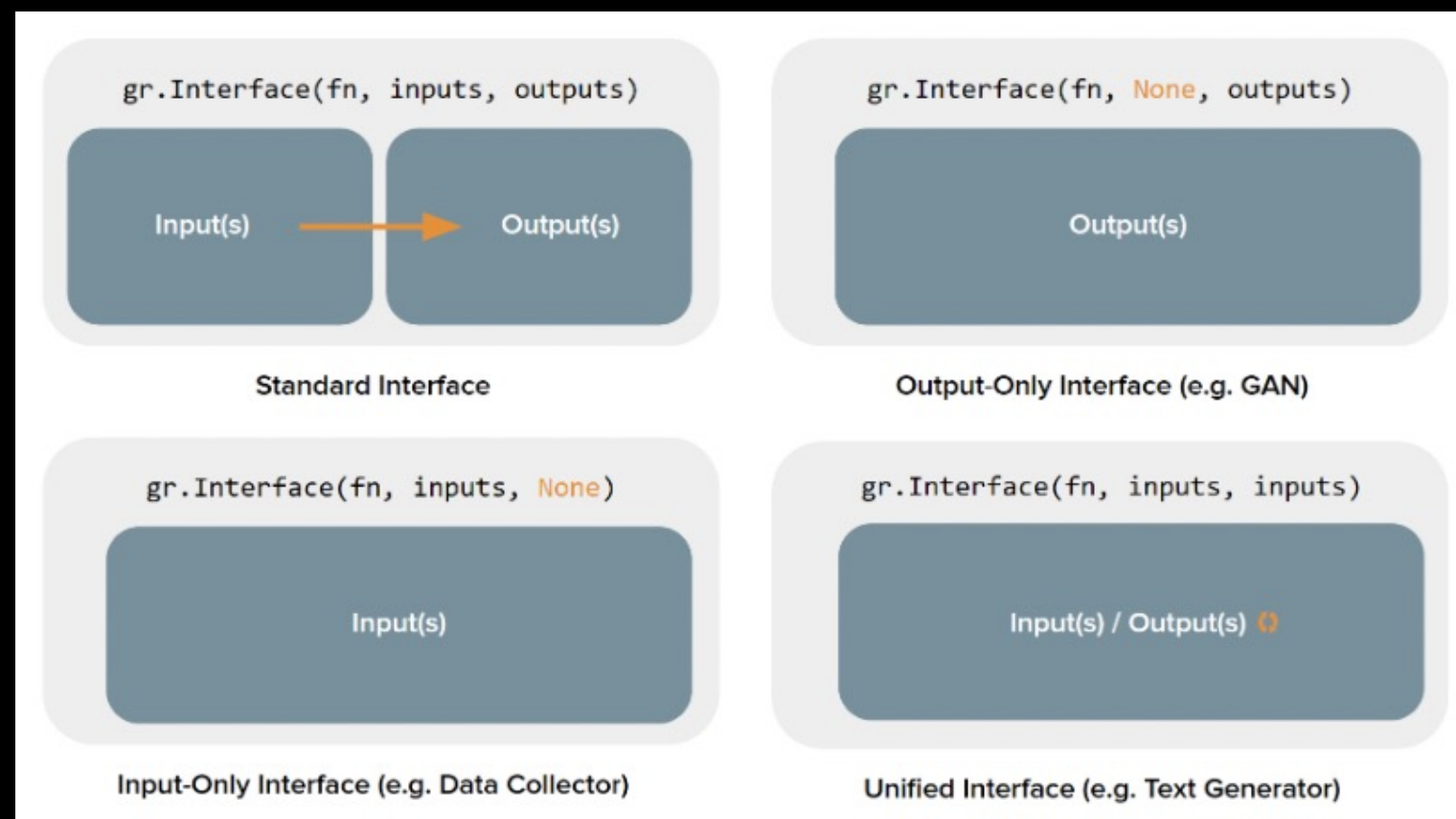
7. Gradio 활용 목표 및 계획



[2장] 데이터와 모델 연동



[3장] Project1. 이미지 분류 웹 앱 만들기 (interface 활용)





[4장] Project2. 텍스트 생성 웹 앱 만들기 (block & component 활용)

microsoft/DialoGPT-large

Textbox

What do you do?

Clear Submit

output 0

Hello!

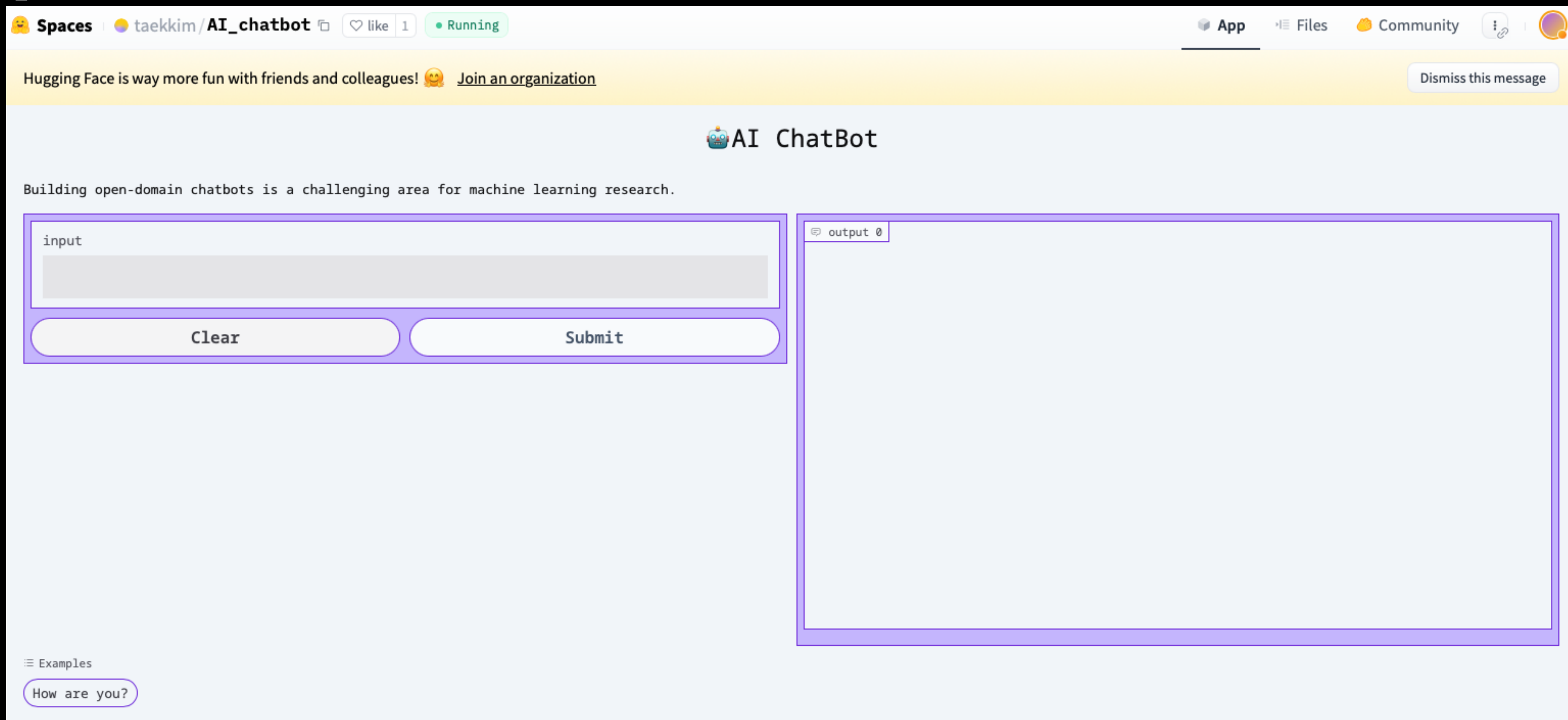
Hiya!

What do you do?

I'm a software developer.



[5장] 프로젝트 공유하기





Gradio를 활용해서
어떤 웹 앱을 만들고 싶나요?

